

Week 3 Day 1 - Gin Framework Basics (< 3 hours)

✅ Your Code Status

Your code is **excellent!** Only 2 minor suggestions at the end.

1. Gin Framework Fundamentals (30 mins)

What is Gin?

Gin is a fast HTTP web framework for Go that makes building APIs easy.

Why Gin?

- Fast (40x faster than some frameworks)
- Easy to learn
- Built-in validation
- Middleware support
- Production-ready

Alternatives:

- `net/http` (standard library - verbose)
- `Echo` (similar to Gin)
- `Fiber` (Express.js-like)

Core Concepts

1. Router

```
go

router := gin.Default() // Creates router with logging & recovery
```

2. Context (`*gin.Context`)

```
go

func handler(c *gin.Context) {
    // c has everything: request data, response methods, params
}
```

3. JSON Responses

```
go
```

```
c.JSON(http.StatusOK, gin.H{
    "message": "Success",
})
```

4. Route Groups

```
go
```

```
userRoutes := router.Group("/users")
{
    userRoutes.GET("", getAllUsers)
    userRoutes.POST("", createUser)
}
```

Your Setup (Perfect!)

```
week3-api/
├── cmd/api/main.go    ← Entry point
├── internal/
│   ├── handlers/user.go ← Business logic
│   └── models/user.go   ← Data structures
└── go.mod
```

Why this structure?

- `cmd/` - Application entry points
- `internal/` - Private application code (can't be imported)
- Separates concerns (models, handlers, main)

2. HTTP Methods & CRUD (45 mins)

HTTP Methods Explained

Method	Purpose	Example
GET	Read data	Get user by ID
POST	Create new	Create user
PUT	Update entire	Update all user fields
PATCH	Update partial	Update only email
DELETE	Remove	Delete user

REST API Conventions

```
GET    /users    → List all users
GET    /users/:id → Get specific user
POST   /users    → Create user
PUT    /users/:id → Update user
DELETE /users/:id → Delete user
```

Your CRUD Implementation

Create (POST):

```
go

func (h *UserHandler) CreateUser(c *gin.Context) {
    var user models.User

    // Bind JSON to struct & validate
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // Save user (assign ID)
    h.mu.Lock()
    user.ID = h.nextID
    h.nextID++
    h.users[user.ID] = user
    h.mu.Unlock()

    // Return created user with 201 status
    c.JSON(http.StatusCreated, user)
}
```

Read (GET):

```
go
```

```
func (h *UserHandler) GetUser(c *gin.Context) {  
    // Get ID from URL path (/users/1)  
    idStr := c.Param("id")  
    id, err := strconv.Atoi(idStr)  
    if err != nil {  
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid user ID"})  
        return  
    }  
  
    // Find user  
    h.mu.RLock()  
    user, exists := h.users[id]  
    h.mu.RUnlock()  
  
    if !exists {  
        c.JSON(http.StatusNotFound, gin.H{"error": "User not found"})  
        return  
    }  
  
    c.JSON(http.StatusOK, user)  
}
```

Update (PUT):

```
go
```

```

func (h *UserHandler) UpdateUser(c *gin.Context) {
    // Parse ID
    id, err := strconv.Atoi(c.Param("id"))
    if err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid user ID"})
        return
    }

    h.mu.Lock()
    defer h.mu.Unlock()

    // Check if user exists
    if _, exists := h.users[id]; !exists {
        c.JSON(http.StatusNotFound, gin.H{"error": "User not found"})
        return
    }

    // Bind new data
    var user models.User
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // Keep same ID, update data
    user.ID = id
    h.users[id] = user

    c.JSON(http.StatusOK, user)
}

```

Delete (DELETE):

```

go

```

```

func (h *UserHandler) DeleteUser(c *gin.Context) {
    id, err := strconv.Atoi(c.Param("id"))
    if err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid user ID"})
        return
    }

    h.mu.Lock()
    defer h.mu.Unlock()

    // Check existence
    if _, exists := h.users[id]; !exists {
        c.JSON(http.StatusNotFound, gin.H{"error": "User not found"})
        return
    }

    // Delete from map
    delete(h.users, id)

    c.JSON(http.StatusOK, gin.H{"message": "User deleted successfully"})
}

```

Key Gin Methods

Getting data from request:

```

go

c.Param("id")           // URL parameter: /users/:id
c.Query("page")         // Query string: ?page=1
c.DefaultQuery("page", "1") // With default value
c.ShouldBindJSON(&user)   // Parse JSON body

```

Sending responses:

```

go

c.JSON(200, data)       // JSON response
c.String(200, "Hello")   // Plain text
c.XML(200, data)         // XML response
c.Redirect(302, "/login") // Redirect

```

Status codes:

```

go

```

```
http.StatusOK           // 200
http.StatusCreated      // 201
http.StatusBadRequest  // 400
http.StatusNotFound     // 404
http.StatusInternalServerError // 500
```

3. Request Validation (45 mins)

Validation Tags

Your model uses **struct tags** for validation:

```
go

type User struct {
    ID   int   `json:"id"`
    Name string `json:"name" binding:"required"`
    Email string `json:"email" binding:"required,email"`
    Age  int   `json:"age" binding:"required,gte=0,lte=130"`
}
```

Tag Breakdown

json:"name" - JSON serialization

```
json

{"name": "Alice"} ← JSON field name
```

binding:"required" - Must be present

```
go

// This fails validation:
{"email": "test@example.com", "age": 25} // Missing name
```

binding:"email" - Must be valid email

```
go

"alice@example.com" ✓
"invalid"           ✗
"@example.com"      ✗
```

binding:"gte=0,lte=130" - Range validation

go

age: 25  (between 0 and 130)

age: -5  (less than 0)

age: 200  (greater than 130)

Common Validation Tags

go

```
type Example struct {  
    // Required field  
    Name string `binding:"required"`  
  
    // Email format  
    Email string `binding:"required,email"`  
  
    // Minimum length  
    Password string `binding:"required,min=8"`  
  
    // Range  
    Age int `binding:"gte=0,lte=150"`  
  
    // One of specific values  
    Status string `binding:"oneof=active inactive pending"`  
  
    // URL format  
    Website string `binding:"url"`  
  
    // Custom validation (alphanumeric)  
    Username string `binding:"required,alphanum"`  
}
```

How Validation Works

1. Request comes in:

bash

POST /users

```
{"name": "Alice", "email": "invalid", "age": 25}
```

2. `ShouldBindJSON` validates:

go

```
if err := c.ShouldBindJSON(&user); err != nil {  
    // err contains validation errors  
    c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})  
    return  
}
```

3. Error response:

```
json  
  
{  
    "error": "Key: 'User.Email' Error:Field validation for 'Email' failed on the 'email' tag"  
}
```

Better Error Messages

Improve validation errors (optional):

```
go
```

```

func (h *UserHandler) CreateUser(c *gin.Context) {
    var user models.User

    if err := c.ShouldBindJSON(&user); err != nil {
        // Parse validation errors
        if validationErrs, ok := err.(validator.ValidationErrors); ok {
            errors := make(map[string]string)
            for _, e := range validationErrs {
                errors[e.Field()] = formatError(e)
            }
            c.JSON(http.StatusBadRequest, gin.H{"errors": errors})
            return
        }
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // ... rest of handler
}

func formatError(e validator.FieldError) string {
    switch e.Tag() {
    case "required":
        return "This field is required"
    case "email":
        return "Invalid email format"
    case "gte":
        return fmt.Sprintf("Must be at least %s", e.Param())
    case "lte":
        return fmt.Sprintf("Must be at most %s", e.Param())
    default:
        return "Invalid value"
    }
}

```

4. Route Groups & Organization (30 mins)

Why Route Groups?

Without groups:

```
go
```

```
router.GET("/users", getAllUsers)
router.GET("/users/:id", getUser)
router.POST("/users", createUser)
router.PUT("/users/:id", updateUser)
router.DELETE("/users/:id", deleteUser)
```

With groups (your code):

```
go

userRoutes := router.Group("/users")
{
    userRoutes.GET("", getAllUsers)
    userRoutes.GET("/:id", getUser)
    userRoutes.POST("", createUser)
    userRoutes.PUT("/:id", updateUser)
    userRoutes.DELETE("/:id", deleteUser)
}
```

Benefits:

- Less repetition (`/users` once)
- Easy to add middleware to group
- Cleaner organization

Multiple Route Groups

```
go
```

```

func main() {
    router := gin.Default()

    // API v1
    v1 := router.Group("/api/v1")
    {
        users := v1.Group("/users")
        {
            users.GET("", getAllUsers)
            users.POST("", createUser)
        }

        posts := v1.Group("/posts")
        {
            posts.GET("", getAllPosts)
            posts.POST("", createPost)
        }
    }

    // API v2 (different structure)
    v2 := router.Group("/api/v2")
    {
        v2.GET("/users", getAllUsersV2)
    }

    router.Run(":8080")
}

```

Results in:

```

GET /api/v1/users
POST /api/v1/users
GET /api/v1/posts
POST /api/v1/posts
GET /api/v2/users

```

5. Testing Your API (20 mins)

Using curl (Command Line)

Create user:

```
bash
```

```
curl -X POST http://localhost:8080/users \  
-H "Content-Type: application/json" \  
-d '{"name":"Alice","email":"alice@example.com","age":25}'
```

Get all users:

```
bash  
  
curl http://localhost:8080/users
```

Get specific user:

```
bash  
  
curl http://localhost:8080/users/1
```

Update user:

```
bash  
  
curl -X PUT http://localhost:8080/users/1 \  
-H "Content-Type: application/json" \  
-d '{"name":"Alice Smith","email":"alice.smith@example.com","age":26}'
```

Delete user:

```
bash  
  
curl -X DELETE http://localhost:8080/users/2
```

Using Postman (GUI)

1. Download Postman (postman.com)
2. Create collection "Week3 API"
3. Add requests:
 - GET `http://localhost:8080/users`
 - POST `http://localhost:8080/users` (Body → raw → JSON)
 - etc.

Using HTTPie (Prettier curl)

```
bash
```

```
# Install: brew install httpie (Mac) or pip install httpie
```

```
# GET
```

```
http :8080/users
```

```
# POST (auto-adds Content-Type: application/json)
```

```
http POST :8080/users name=Alice email=alice@example.com age=25
```

```
# PUT
```

```
http PUT :8080/users/1 name="Alice Smith" email=alice.smith@example.com age=26
```

```
# DELETE
```

```
http DELETE :8080/users/1
```

Practice Exercise (30 mins)

Task: Add Product CRUD

Create `internal/models/product.go`:

```
go

package models

type Product struct {
    ID      int    `json:"id"`
    Name     string `json:"name" binding:"required"`
    Description string `json:"description"`
    Price    float64 `json:"price" binding:"required,gt=0"`
    InStock  bool   `json:"in_stock"`
}
```

Create `internal/handlers/product.go`:

package handlers

```
import (  
    "net/http"  
    "strconv"  
    "sync"  
  
    "week3-api/internal/models"  
    "github.com/gin-gonic/gin"  
)
```

```
type ProductHandler struct {  
    products map[int]models.Product  
    nextID   int  
    mu       sync.RWMutex  
}
```

```
func NewProductHandler() *ProductHandler {  
    return &ProductHandler{  
        products: make(map[int]models.Product),  
        nextID:    1,  
    }  
}
```

// TODO: Implement these methods (copy pattern from UserHandler):

*// - GetAllProducts(c *gin.Context)*

*// - GetProduct(c *gin.Context)*

*// - CreateProduct(c *gin.Context)*

*// - UpdateProduct(c *gin.Context)*

*// - DeleteProduct(c *gin.Context)*

Add to **main.go**:

```
go  
  
productHandler := handlers.NewProductHandler()  
  
productRoutes := router.Group("/products")  
{  
    productRoutes.GET("", productHandler.GetAllProducts)  
    productRoutes.GET("/:id", productHandler.GetProduct)  
    productRoutes.POST("", productHandler.CreateProduct)  
    productRoutes.PUT("/:id", productHandler.UpdateProduct)  
    productRoutes.DELETE("/:id", productHandler.DeleteProduct)  
}
```

Test:

```
bash

# Create product
curl -X POST http://localhost:8080/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop","description":"Gaming laptop","price":1299.99,"in_stock":true}'

# List products
curl http://localhost:8080/products
```

 Quick Reference

HTTP Status Codes

Code	Meaning	When to use
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST (created resource)
400	Bad Request	Invalid input/validation error
404	Not Found	Resource doesn't exist
500	Server Error	Unexpected server error

Gin Context Methods

```
go

// Request data
c.Param("id")           // Path parameter
c.Query("page")         // Query string
c.ShouldBindJSON(&data) // Parse JSON

// Responses
c.JSON(200, data)        // JSON
c.String(200, "text")    // Plain text
c.Status(204)            // Status only (no body)

// Headers
c.GetHeader("Authorization")
c.Header("X-Custom", "value")
```

Validation Tags

```
go
```

```
binding:"required"    // Must be present
binding:"email"       // Email format
binding:"min=8"        // Minimum length/value
binding:"max=100"      // Maximum length/value
binding:"gte=0"        // Greater than or equal
binding:"lte=150"      // Less than or equal
binding:"oneof=a b c"  // One of values
```

Minor Improvements

1. Add Error Helper Function

Create `internal/handlers/response.go`:

```
go

package handlers

import "github.com/gin-gonic/gin"

func ErrorResponse(c *gin.Context, code int, message string) {
    c.JSON(code, gin.H{"error": message})
}

func successResponse(c *gin.Context, code int, data interface{}) {
    c.JSON(code, data)
}
```

Use in handlers:

```
go

func (h *UserHandler) GetUser(c *gin.Context) {
    id, err := strconv.Atoi(c.Param("id"))
    if err != nil {
        ErrorResponse(c, http.StatusBadRequest, "Invalid user ID")
        return
    }

    // ... rest of code
}
```

2. Add Graceful Shutdown

Update `main.go`:


```
import (  
    "context"  
    "log"  
    "net/http"  
    "os"  
    "os/signal"  
    "syscall"  
    "time"  
)  
  
func main() {  
    router := gin.Default()  
  
    // ... setup routes  
  
    // Create server  
    srv := &http.Server{  
        Addr:    ":8080",  
        Handler: router,  
    }  
  
    // Start server in goroutine  
    go func() {  
        if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {  
            log.Fatalf("Server error: %v", err)  
        }  
    }()  
  
    // Wait for interrupt signal  
    quit := make(chan os.Signal, 1)  
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)  
    <-quit  
  
    log.Println("Shutting down server...")  
  
    // 5 second timeout for shutdown  
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)  
    defer cancel()  
  
    if err := srv.Shutdown(ctx); err != nil {  
        log.Fatal("Server forced to shutdown:", err)  
    }  
  
    log.Println("Server exited")  
}
```

✓ Day 1 Checklist

After 3 hours, you should:

- ☐ Understand Gin router and context
 - ☐ Implemented GET, POST, PUT, DELETE
 - ☐ Request validation working with tags
 - ☐ CRUD operations for users complete
 - ☐ Can test API with curl/Postman
 - ☐ (Bonus) Completed product CRUD exercise
-

🔗 Optional Reading

Gin documentation:

- Official docs: gin-gonic.com/docs

Validation:

- Package: github.com/go-playground/validator
- Validation tags: pkg.go.dev/github.com/go-playground/validator/v10

REST API design:

- Article: "Best Practices for REST API Design"

Tools:

- Postman: postman.com
 - HTTPie: httpie.io
-

🎯 Key Takeaways

1. **Gin makes APIs easy** - `router.GET()`, `c.JSON()`, done!
2. **Context has everything** - `c.Param()`, `c.Query()`, `c.ShouldBindJSON()`
3. **Validation is automatic** - Use struct tags, Gin handles it
4. **Route groups organize** - `/api/v1` prefix for all routes
5. **Status codes matter** - 200 OK, 201 Created, 400 Bad Request, 404 Not Found

Your code is production-ready for Day 1! Tomorrow: Middleware and error handling.